

02 — System Architecture

Complete documentation suite (Markdown → PDF with Mermaid diagrams)

Generated: 2026-09-22

Source: repository `docs/` directory

Diagrams rendered with Mermaid 11

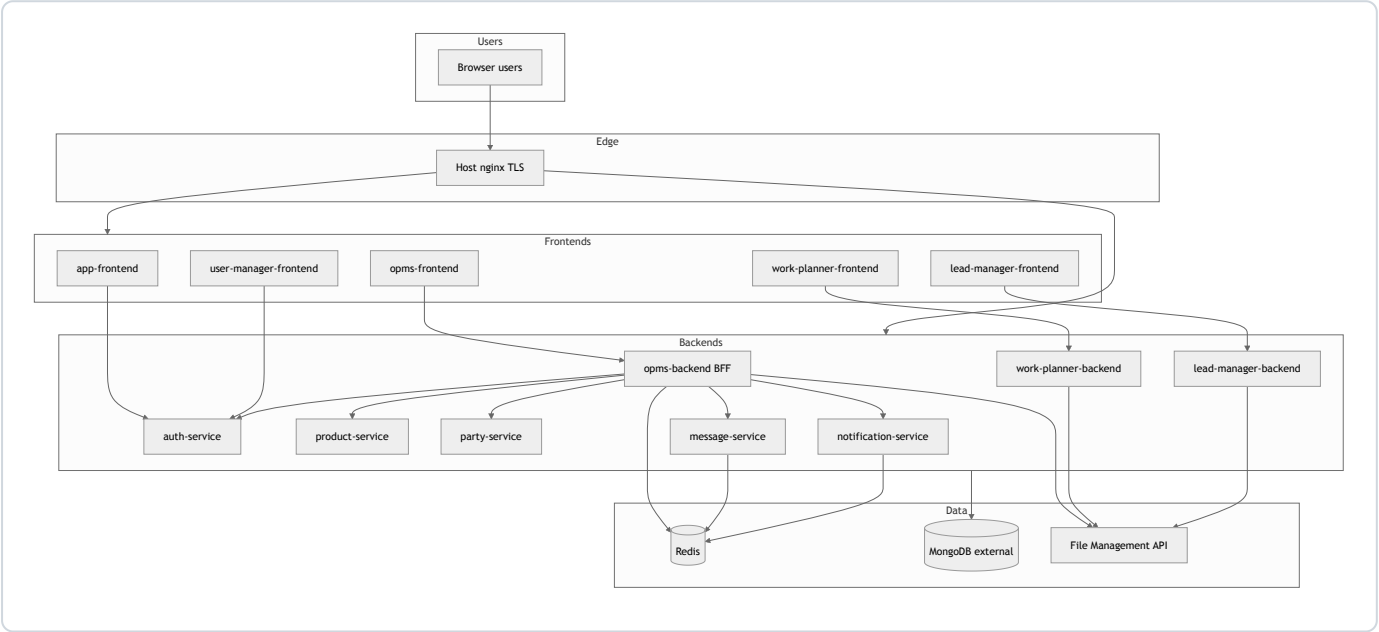
Table of Contents

- [02-architecture/01-system-architecture.md](#)
- [02-architecture/02-component-architecture.md](#)
- [02-architecture/03-frontend-architecture.md](#)
- [02-architecture/04-backend-architecture.md](#)
- [02-architecture/05-service-architecture.md](#)
- [02-architecture/06-data-flow.md](#)
- [02-architecture/07-authentication-authorization.md](#)
- [02-architecture/08-integration-architecture.md](#)
- [02-architecture/09-infrastructure-architecture.md](#)
- [02-architecture/10-deployment-architecture.md](#)
- [02-architecture/11-architecture-decisions.md](#)
- [02-architecture/12-adr/ADR-001-microservice-split.md](#)
- [02-architecture/12-adr/ADR-002-opms-backend-bff.md](#)
- [02-architecture/12-adr/ADR-003-shared-mongodb.md](#)
- [02-architecture/12-adr/ADR-004-jwt-portal-rbac.md](#)
- [02-architecture/12-adr/ADR-005-bullmq-redis.md](#)
- [02-architecture/12-adr/ADR-006-micro-frontends-sso.md](#)
- [02-architecture/12-adr/ADR-007-external-file-api.md](#)
- [02-architecture/12-adr/ADR-008-unified-order-approval.md](#)

System Architecture

Context

OPMS is a **Docker Compose–orchestrated microservice + micro-frontend** system. Browsers talk to Next.js apps and/or public API hostnames (via host nginx in production). Backends share a **MongoDB** database (external) and **Redis** (in Compose) for queues.



Runtime ports (Compose defaults)

Service	Port
opms-backend	7001
opms-frontend	7002
auth-service	7003
user-manager-frontend	7004
product-service	7005
party-service	7006
work-planner-backend	7007
work-planner-frontend	7008
lead-manager-backend	7009
lead-manager-frontend	7010
message-service	7011
notification-service	7012

Service	Port
app-frontend	7013
redis	7014→6379

Sources: `docker-compose.yml` , `DOCKER.md`

Architectural style

Style element	Application
Microservices	Domain-split Node/Express services
BFF / API gateway-lite	opms-backend proxies auth/product/party/message/notification
Micro-frontends	Separate Next.js apps; SSO hub launches peers
Shared database	Multiple services point at same Mongo URI (Inferred from env sharing; collection ownership logical)
Async messaging	BullMQ on Redis (not RabbitMQ)

Related

- [Component Architecture](#)
- [Deployment Architecture](#)

Component Architecture

Backend components

Component	Responsibility
auth-service	Login, JWT, users, roles, departments, portals, company info; seeds defaults
opms-backend	Order domain, workflow, finance/dispatch/fleet/transport planner, dashboards; proxies sibling APIs; BullMQ workers
product-service	Product catalog, groups/subgroups/brands/manufacturers, kits, batches
party-service	Parties, zones, party-product mappings/rates
lead-manager-backend	Leads, quotations, lead masters, follow-up cron; proxies messaging
work-planner-backend	Work plans, visits, works, expenses, settings; proxies parties/leads/messaging
message-service	Email/WhatsApp/messages queues
notification-service	In-app notifications, SSE, web push

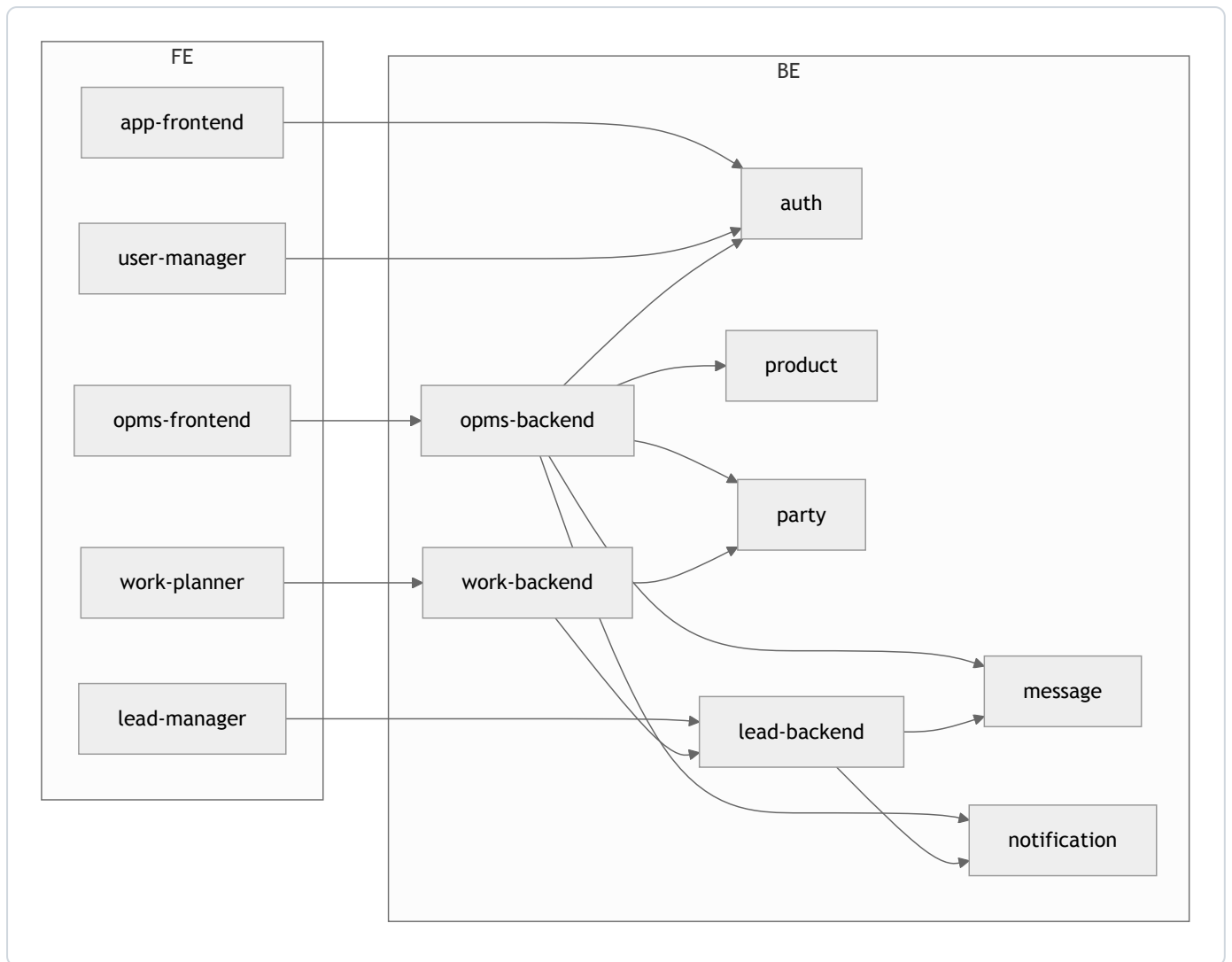
Frontend components

Component	Responsibility
app-frontend	Login + portal launcher SSO hub
opms-frontend	Multi-role OPMS workspace
user-manager-frontend	Identity administration UI
lead-manager-frontend	CRM UI
work-planner-frontend	Field planning UI

Shared technical components

- Mongoose models / `mongoRegistry.js` (duplicated across services)
- `softDelete.plugin.js`
- JWT auth middlewares
- Multer uploads → File Management API
- Swagger (opms-backend only)

Component diagram

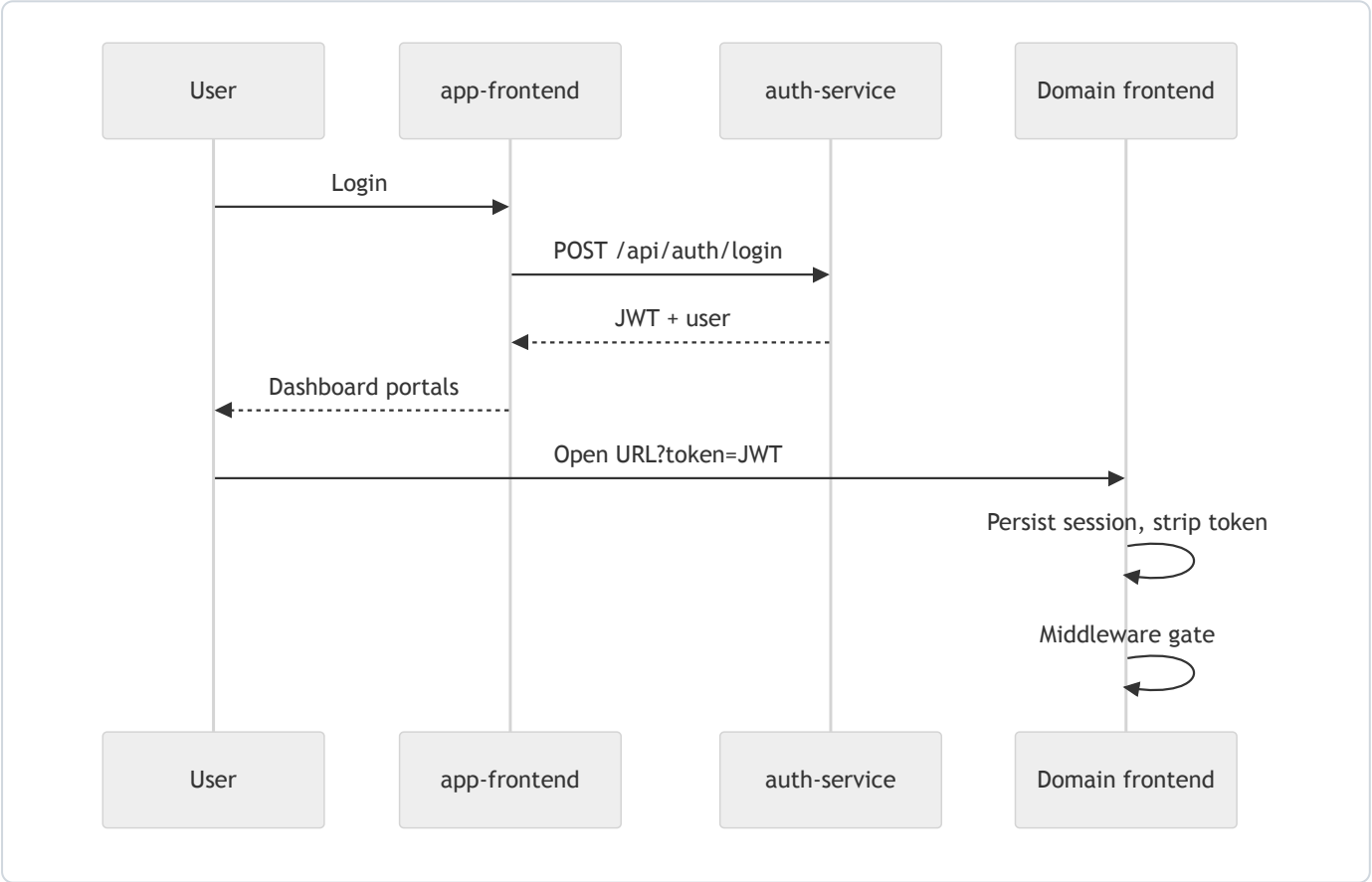


Frontend Architecture

Stack (all five apps)

- **Next.js 16** App Router
- **React 19**
- **Redux Toolkit** + **RTK Query**
- **Tailwind CSS 4**
- **lucide-react, sonner**

Hub-and-spoke



Session storage patterns

App	localStorage key (primary)	Cookies
app-frontend	shakti.app.session	shakti_session , medica_session
opms-frontend	medica.auth	medica_session , medica_opms_roles
user-manager	shakti.user_manager.session	shared session cookies
lead-manager	shakti.lead_manager.session	shared

App	localStorage key (primary)	Cookies
work-planner	shakti.work_planner.session	shared

OPMS shell

- Dynamic route `/[portal]/[...rest]`
- Shared Sidebar/Topbar
- Portal allowlists for paths
- Deep links to Lead Manager / User Manager iframe for some paths

API clients

Frontend	Primary base
app-frontend	NEXT_PUBLIC_AUTH_SERVICE_URL
opms-frontend	NEXT_PUBLIC_API_ORIGIN → opms-backend
user-manager	auth-service URL
lead-manager	NEXT_PUBLIC_LEAD_MANAGER_SERVICE_URL
work-planner	NEXT_PUBLIC_WORK_PLANNER_SERVICE_URL

Note: `NEXT_PUBLIC_*` values are **build-time** baked into images.

Backend Architecture

Common pattern

Each API service:

1. `src/server.js` — connect Mongo, optional queues/workers, listen
2. `src/app.js` — Express app, CORS, JSON limit, mount routers, `/health`
3. `src/modules/*` — routes → controllers → services
4. `src/data/mongoRegistry.js` or `src/models/*` — Mongoose schemas
5. `src/middlewares/*` — auth / portal gates

opms-backend as BFF

Mounts local domain routers under `/api/*` and **HTTP-proxies**:

- `/api/auth` , `/api/users` , `/api/company-info` , `/api/portals` , `/api/departments` → auth-service
- `/api/products` , `product-*` → product-service
- `/api/parties` , `zones` , `party-*` → party-service
- `/api/notifications` , `push` → notification-service
- `/api/messages` , `emails` , `auto-emails` , `communication` → message-service

Source: `opms-backend/src/app.js`

Queues / workers (opms-backend)

BullMQ queues under `src/queues/` (orders, orderApproval, dispatches, workflow, reports) with workers in `src/workers/` .

message-service: `messages` , `autoEmails`

notification-service: `notifications`

Auth middleware pattern

1. Global optional JWT populate
2. Route `requireAuth`
3. Portal gate (`requireOpmsAccess` / lead / work planner)

OpenAPI

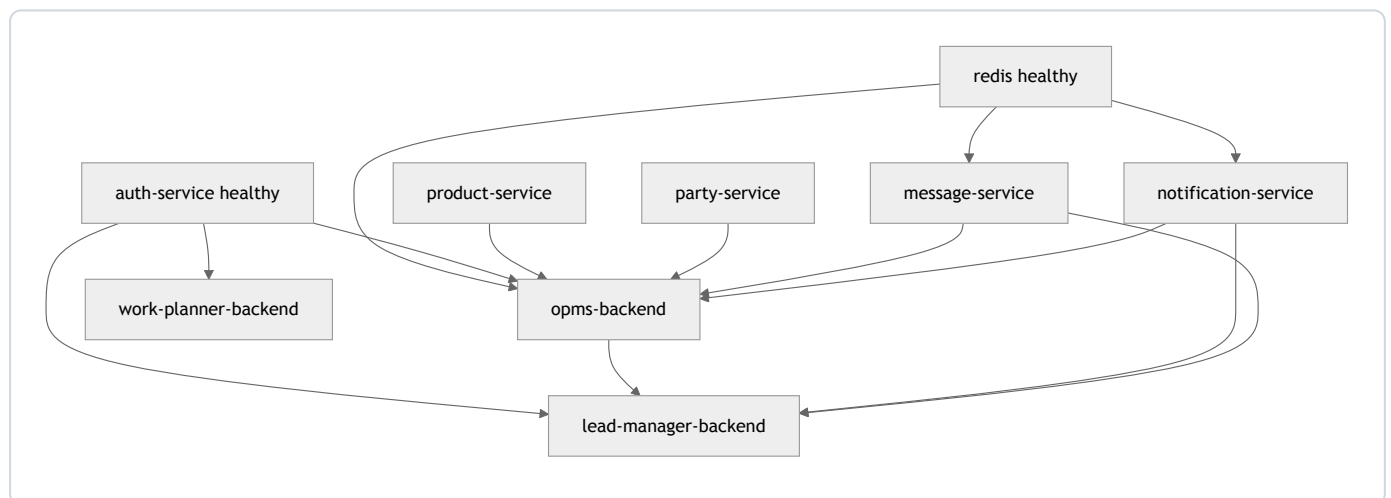
Only opms-backend exposes Swagger UI at `/api-docs` .

Service Architecture

Service communication matrix

From	To	Mechanism
Frontends	Owning backends	HTTPS/HTTP REST + Bearer JWT
opms-backend	auth/product/party/message/notification	HTTP proxy / axios
work-planner-backend	party, lead-manager, message, notification	HTTP proxy
lead-manager-backend	message, notification	HTTP proxy
auth-service	notification-service	proxy /api/notifications
Queue producers	Redis	BullMQ
All APIs	MongoDB	Mongoose
Upload flows	File Management API	HTTP + API key

Startup dependencies (compose)

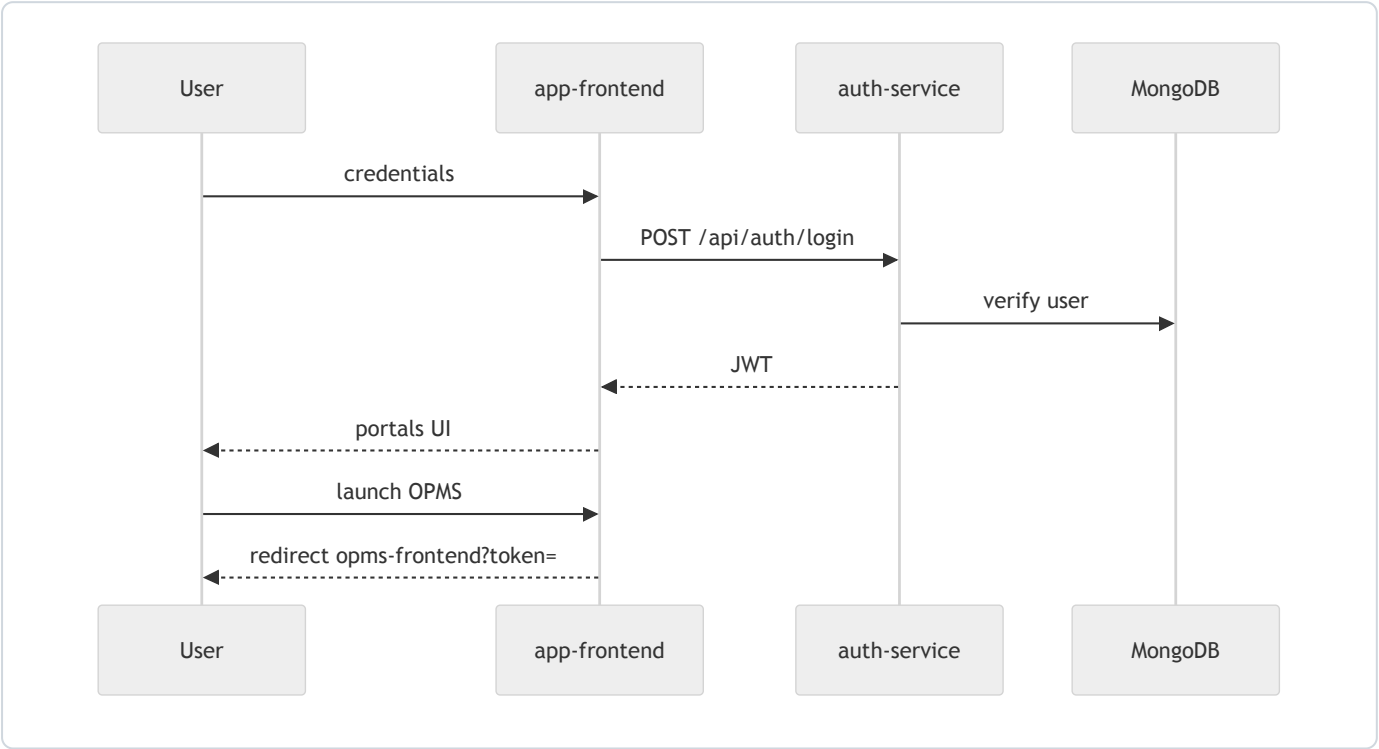


Health

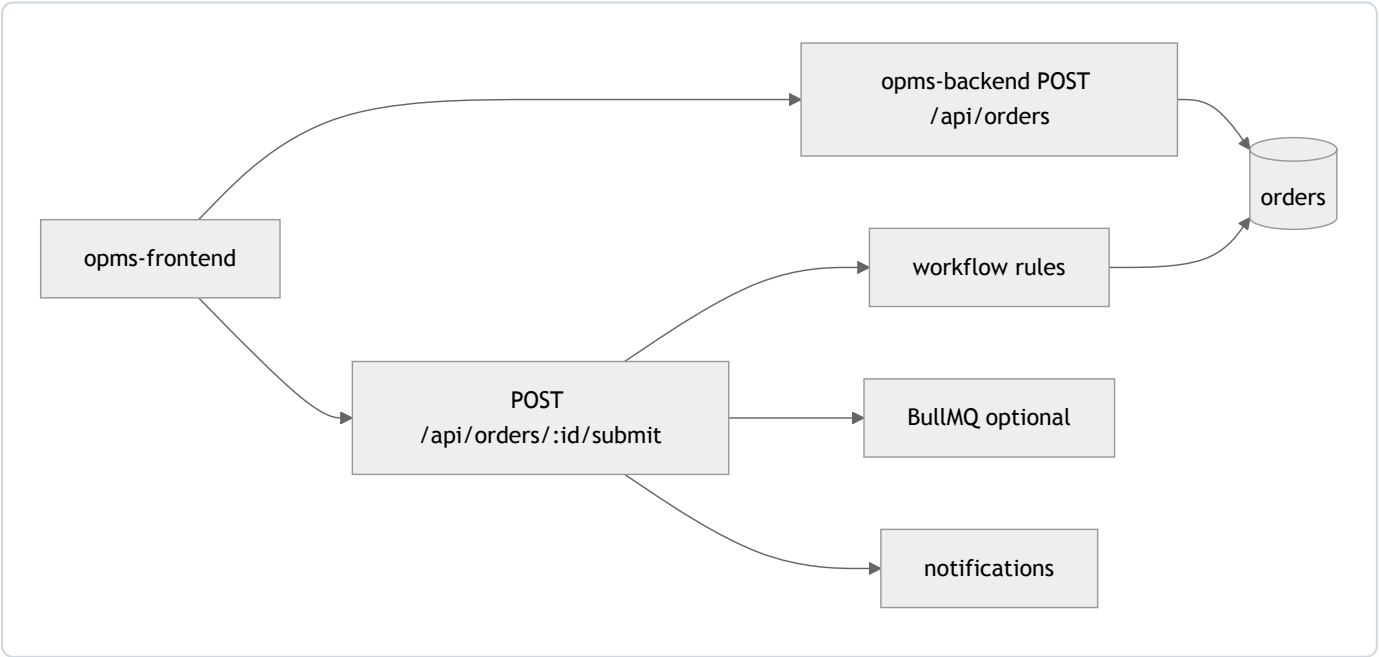
Each backend exposes `GET /health`. Frontends: **no** compose healthchecks in base file.

Data Flow

1. Login and portal launch



2. Create and submit order



3. Approval path (simplified)

Sales/Admin create/update OrderApproval → send-to-finance → finance approve/amend → send-to-account → account approve → order status advances per transitions → dispatch created → transport/delivery.

4. Messaging

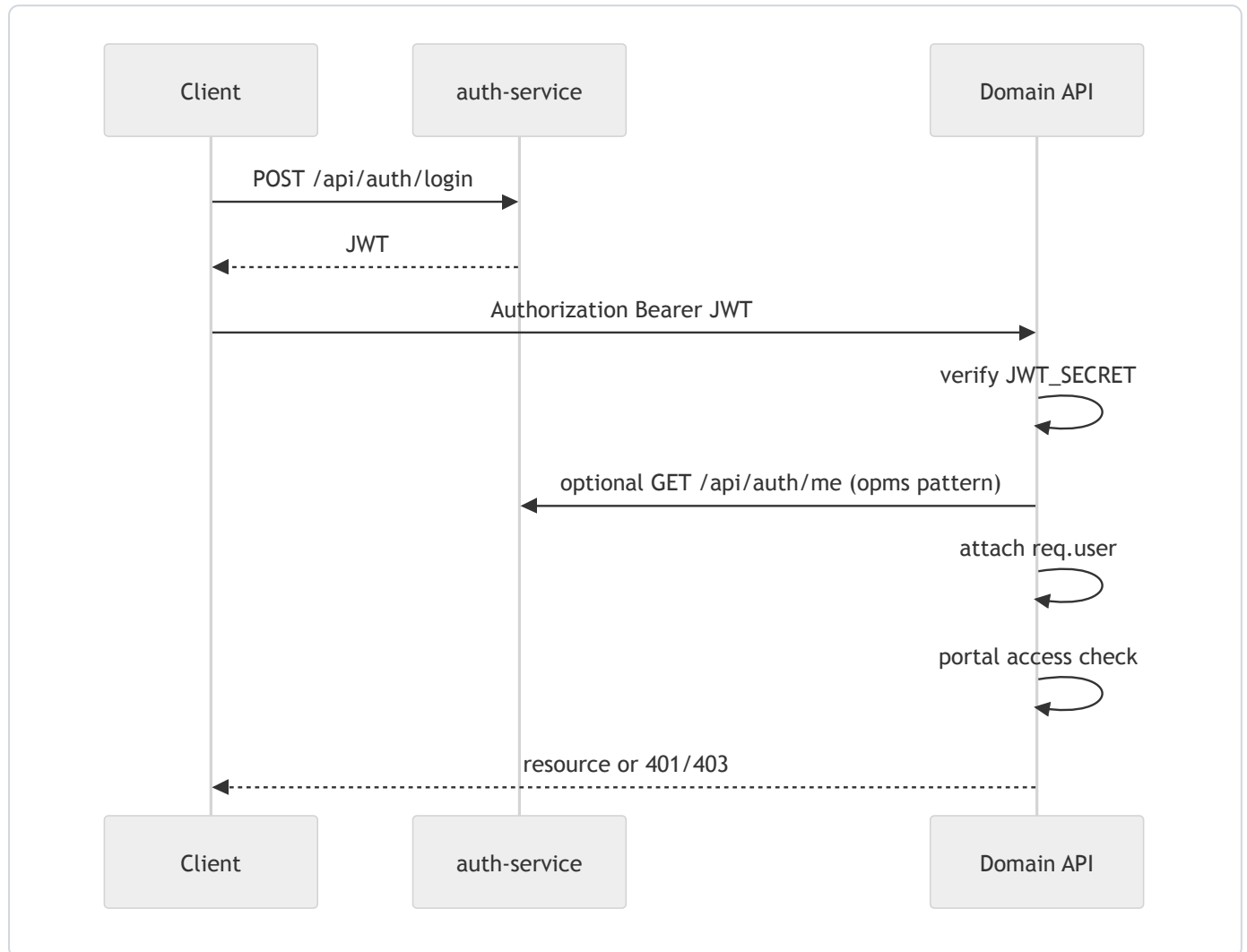
Domain event/API → message-service enqueue → worker sends via Graph/SMTP/WhatsApp → Message document status updated.

5. File upload

Client → attachments route (multer) → File Management API → Attachment metadata in Mongo → view/download via `/api/files/:fileId/*` redirect.

Authentication & Authorization

Authentication flow



Token

- Signed with `JWT_SECRET`
- Expiry `JWT_EXPIRES_IN` (example default `7d`)
- Optional `MASTER_PASSWORD` break-glass (**dangerous if set in prod**)

Authorization layers

1. **Authenticated** — valid JWT
2. **Portal membership** — `portals[]` contains required `portal_code`
3. **access_roles** — role string must match portal gate (or admin bypass where coded)
4. **Handler-level checks** — department / role for specific actions (workflow rules, lead admin routes)

5. **Soft-delete permission** — authenticated (+ opms) for trash/restore on some routes; swagger mentions permission codes that may be aspirational vs portal model — **Requires confirmation** for fine-grained permission codes vs portal roles

OPMS roles

super_admin | admin | sales | finance | account | dispatch

Cross-portal

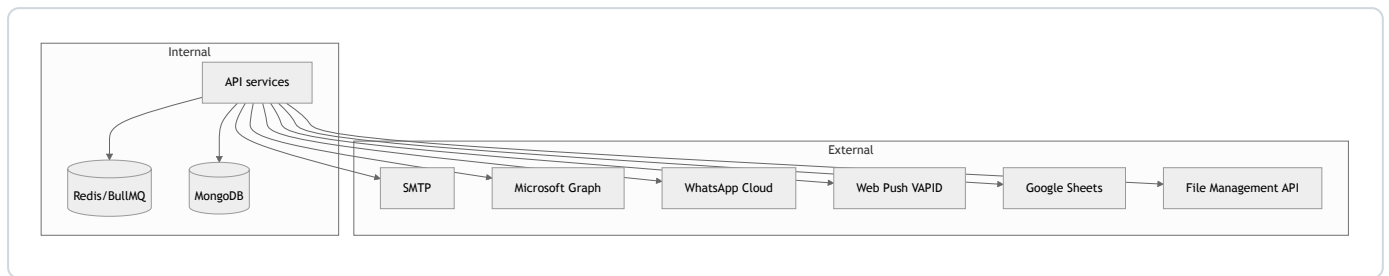
Lead list may allow work_planner access in some routes (`requireLeadManagerOrWorkPlannerAccess`).

Public / weakly protected surfaces (document carefully)

Surface	Notes
POST /api/auth/login	Public
GET /api/company-info	Public on auth-service
Google Sheet webhooks	Secret header/param
WhatsApp webhooks	Public verify/receive
notification internal POSTs	No requireAuth — service trust
opms terms-and-conditions router	No requireAuth on router
lead attachments router	No requireAuth on router
work-planner attachment view	Public view path before auth

See [Security](#).

Integration Architecture



Email

Preferred: Microsoft Graph sendMail when configured; SMTP fallback via nodemailer.

WhatsApp

Cloud API templates + webhook endpoints on message-service.

Files

All binary storage delegated to external File Management API (historically MinIO-backed). Local `minio-data/` is leftover, not compose-managed.

Sheets

Inbound webhooks protected by `GOOGLE_SHEET_WEBHOOK_SECRET` .

Infrastructure Architecture

Compose topology

Sources: `docker-compose.yml` , `docker-compose.dev.yml` , `docker-compose.prod.yml` , `DOCKER.md`

- Single default bridge network
- Named volume: `redis_data`
- `extra_hosts: host.docker.internal:host-gateway` for host Mongo
- Images: `node:20-alpine`

External dependencies

- MongoDB (Atlas or host)
- File Management API
- Optional SaaS: Graph, WhatsApp, SMTP

Nginx

Host-installed reverse proxy; configs in `nginx/sites-available/opms` and `medica` .

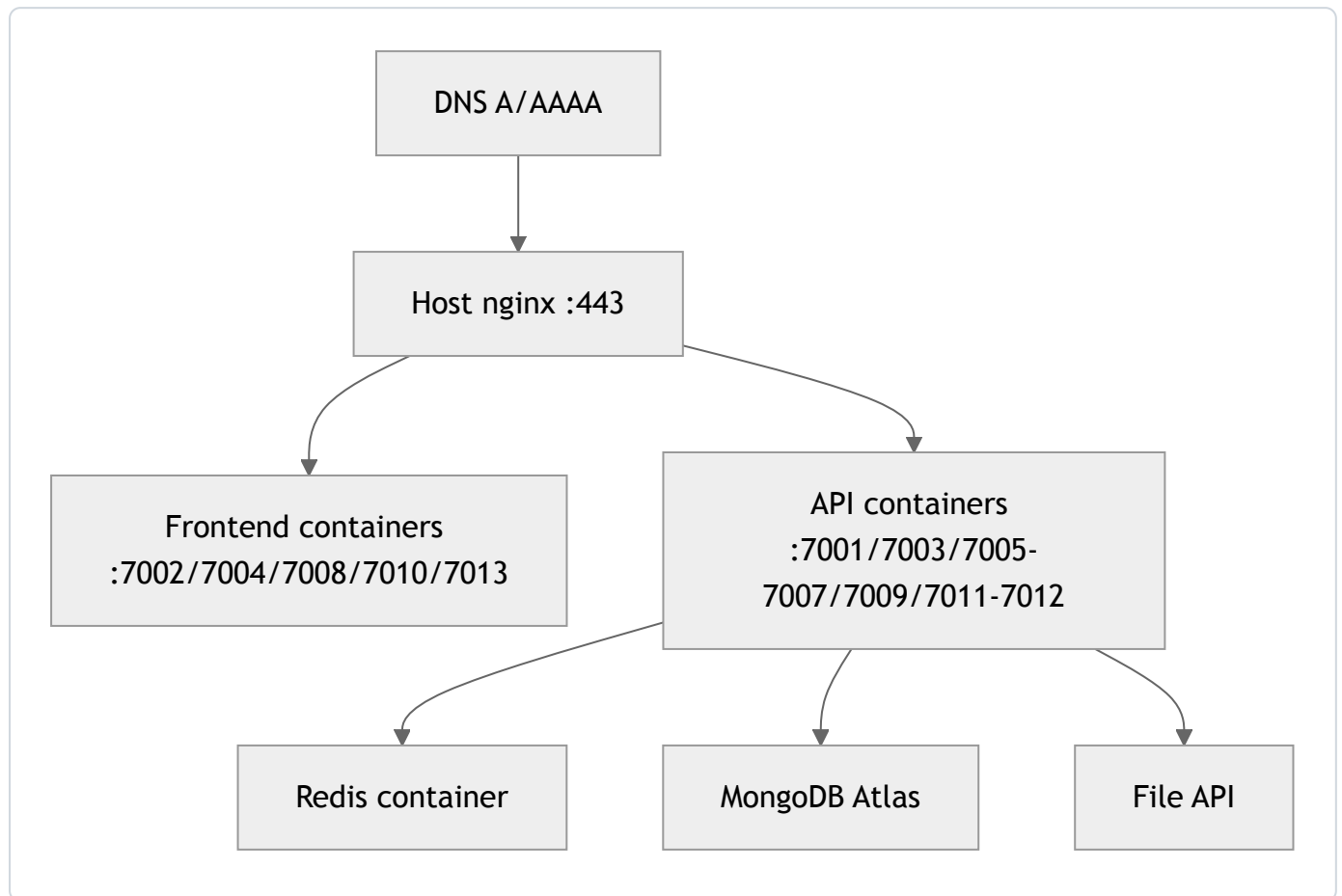
Dual tenant

Tenant	FE/API ports	DNS
OPMS/spspl	7001–7014	<code>*.spspl.com</code>
Medica	5001–5013 (nginx upstreams)	<code>*.medicaent.in</code>

What is not in infra-as-code here

- CI/CD
- Kubernetes/Helm
- Managed monitoring
- Mongo/Redis backup automation

Deployment Architecture



Recommended path (from DOCKER.md)

1. Configure `.env.docker`
2. `docker compose --env-file .env.docker up --build -d`
3. Install nginx site + certbot SANs
4. Point DNS to host

Prod compose file caveat

`docker-compose.prod.yml` is a **partial** stack — prefer full `docker-compose.yml` unless ops intentionally runs a subset (**Verified** by file contents).

Rollback

Not identified as automated. Typical approach: redeploy previous image tags / git revision and `compose up` — **Requires confirmation** of image registry strategy (compose builds locally by default).

Architecture Decisions (Index)

Historical rationale is often **not recorded** in-repo. ADRs below capture decisions **evidenced by the current implementation**, with unknown history called out.

ADR	Title
ADR-001	Split domain into multiple Express services
ADR-002	opms-backend as BFF/proxy
ADR-003	Shared MongoDB across services
ADR-004	JWT + portal access_roles RBAC
ADR-005	BullMQ on Redis instead of broker
ADR-006	Micro-frontends with SSO hub
ADR-007	External file management API
ADR-008	Unified OrderApproval model

ADR-001 — Microservice split by domain

Status: Accepted (evidenced by repo structure)

Date: Not identified in repository

Context: Large order/CRM/field/ops domain with multiple UIs.

Decision: Separate Node services: auth, opms, product, party, lead-manager, work-planner, message, notification.

Alternatives: Modular monolith; serverless functions — **Not evidenced.**

Consequences: Independent deploy/scale; duplicated mongoose registries; operational complexity.

Evidence: Top-level service directories + compose services.

ADR-002 — opms-backend as BFF/proxy

Status: Accepted (evidenced)

Date: Not identified

Context: OPMS UI needs one API origin for many resources.

Decision: Proxy auth/product/party/message/notification under opms-backend `/api/*`.

Alternatives: Frontend calls each service directly only — partially used by other MFEs.

Consequences: Single CORS/API origin for OPMS; coupling/latency through proxy.

Evidence: `opms-backend/src/app.js`

ADR-003 — Shared MongoDB

Status: Accepted (inferred from shared URI + mirrored registries)

Date: Not identified

Context: Cross-service references (User, Party, Order, Lead).

Decision: Services connect to same Mongo deployment; schemas registered per service.

Alternatives: DB-per-service with sync APIs — not implemented.

Consequences: Simple joins by ObjectId; schema drift risk across copied registries.

Evidence: Shared `MONGO_URI` in compose env; parallel `mongoRegistry.js` files.

ADR-004 — JWT with portal access_roles

Status: Accepted

Date: Not identified

Context: Multi-app access control beyond department alone.

Decision: Authorize via `user.portals[].portal_code + access_roles`.

Alternatives: Permission-code RBAC only (Permission model exists in opms registry but live gates are portal-centric).

Consequences: Flexible multi-portal users; seed gap for non-opms portals.

Evidence: Portal middlewares across services.

ADR-005 — BullMQ on Redis

Status: Accepted

Date: Not identified

Context: Async notifications/messages/order jobs.

Decision: Use Redis + BullMQ; do not run RabbitMQ.

Alternatives: RabbitMQ/Kafka — absent.

Consequences: Simple compose dependency; Redis becomes critical path.

Evidence: redis service; bullmq dependencies; queue folders.

ADR-006 — Micro-frontends with SSO hub

Status: Accepted

Date: Not identified

Context: Distinct UX domains (ops, CRM, plans, admin).

Decision: Separate Next apps; app-frontend launches with `?token=` .

Alternatives: Single SPA with route modules — not chosen.

Consequences: Independent releases; duplicated shell/auth utilities; cookie/localStorage key proliferation.

Evidence: Five frontend packages + SSO bootstrap components.

ADR-007 — External file management API

Status: Accepted

Date: Not identified

Context: Attachments for orders/leads/expenses.

Decision: Store binaries via external File Management API; persist metadata in Mongo Attachment.

Alternatives: In-compose MinIO (leftover data dir only).

Consequences: Decoupled storage; runtime dependency on external API key/URL.

Evidence: `FILE_MANAGEMENT_API_*` ; files routes.

ADR-008 — Unified OrderApproval

Status: Accepted

Date: Referenced in `implementation_plan.md`

Context: Separate admin/finance approval models were hard to maintain.

Decision: Single `OrderApproval` (+ `OrderAmendmentUser`) with signature flags.

Alternatives: Keep separate collections — rejected per plan.

Consequences: Cleaner schema; API surface consolidated under `/api/order-approvals`.

Evidence: `implementation_plan.md`; `orderApproval` module; `mongoRegistry`.